ПРАКТИЧЕСКАЯ РАБОТА № 3

Установка окружения и первый запуск проекта

МДК.04.01 Внедрение, поддержка компьютерных систем и обеспечение качества функционирования компьютерных систем

1. Цель работы

Настроить рабочее место под Windows, создать каркас Django-проекта в VS Code, подключить базу данных и кэш, выполнить первый запуск информационной системы «Медицинский центр «Медика»».

2. Состав программного обеспечения

Компонент	Версия	Назначение
Python	3.12	интерпретатор, виртуальное окружение
Django	5.1.4	веб-фреймворк
Django REST Framework	3.15.2	REST API
PostgreSQL	16	СУБД (в учебном запуске — SQLite)
Redis / Memurai	5.2	кэш и очередь задач
Git	2.45	контроль версий проекта

3. Ход работы

3.1. Создание проекта и виртуального окружения

В проводнике создаётся папка проекта, она открывается в VS Code. Виртуальное окружение изолирует зависимости проекта от системного Python.

```
mkdir C:\study\medcenter
cd C:\study\medcenter

python -m venv .venv
.venv\Scripts\activate

python --version
pip install --upgrade pip
```

3.2. Файл зависимостей

Все зависимости фиксируются в requirements.txt с точными версиями — это делает сборку воспроизводимой на другом компьютере.

```
Django==5.1.4
djangorestframework==3.15.2
djangorestframework-simplejwt==5.3.1
psycopg2-binary==2.9.10
python-dotenv==1.0.1

pip install -r requirements.txt
```

3.3. Создание каркаса Django-проекта

```
django-admin startproject config .
python manage.py startapp accounts
mkdir apps
python manage.py startapp catalog apps/catalog
python manage.py startapp clients apps/clients
python manage.py startapp orders apps/orders
python manage.py startapp reports apps/reports
```

Структура проекта после создания каркаса:

```
medcenter/
├── apps/
│   ├── accounts/   — пользователи и роли
│   ├── catalog/    — специализация и услуга
│   ├── clients/    — клиенты
│   ├── orders/     — заказы
│   └── reports/    — отчёты
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── .env.example
├── manage.py
└── requirements.txt
```

3.4. Настройка переменных окружения

Секретный ключ и параметры подключения к базе данных не хранятся в коде — они выносятся в файл .env, а в репозиторий попадает только .env.example.

```
# .env.example
SECRET_KEY=change-me
DEBUG=True
DB_NAME=study
DB_USER=postgres
DB_PASSWORD=postgres
DB_HOST=127.0.0.1
DB_PORT=5432
```

3.5. Создание базы данных

База данных создаётся в psql, после чего параметры подключения прописываются в .env. В учебном варианте допускается SQLite — проект стартует без установки СУБД, а настройка PostgreSQL остаётся закомментированной в settings.py.

```
psql -U postgres
CREATE DATABASE study ENCODING 'UTF8';
\l
\q
```

3.6. Первый запуск

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver

# проверка
System check identified no issues (0 silenced).
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

После запуска административная панель доступна по адресу http://127.0.0.1:8000/admin/, API — по адресу http://127.0.0.1:8000/api/. Первая проверка показывает, что каркас проекта собран корректно и подключение к базе данных работает.

3.7. Проверка работоспособности

- админка открывается, вход выполнен под учётной записью admin;
- созданы справочники: 3 специализации, 6 услуг;
- API возвращает список услуг в формате JSON;
- эндпоинт /api/reports/kpi/ отдаёт рассчитанные показатели.

4. Вывод

Рабочее место настроено: установлен Python 3.12, создано виртуальное окружение, собран каркас Django-проекта из пяти приложений, настроены переменные окружения и база данных. Проект запускается и отдаёт административную панель и API — можно переходить к разработке модулей.